

Ideation

1. Problem Statement

Current online learning systems suffer from "data fragmentation" and "reactive teaching".

- **Late Intervention:** Tutors only realize a student is struggling after they have already failed or dropped out.
- **Information Silos:** Student activity, quiz data, and chatbot logs are scattered across different tools.
- **Manual Overhead:** Communication is often manual and delayed, making it impossible to support large cohorts effectively.
- **Invisible Struggle:** Students lose interest or get stuck on topics without the tutor's knowledge.

2. The Strategic Solution: "The Three Eyes"

TRI NETRA turns raw behavioral telemetry into actionable teaching insights through three core pillars:

- **Visibility:** A real-time view of student progress and engagement.
- **Early Detection:** Algorithmic identification of "Drifting" or "Friction" before it leads to failure.
- **Actionability:** Integrated AI tools to intervene immediately and effectively.

3. Core Use Cases & Personas

The Proactive Tutor (Primary User)

- **Live Monitoring:** Tracking video play/pause events and quiz attempts in real time.
- **Friction Diagnosis:** Identifying students who are "stuck" due to multiple quiz failures or repeated content views.
- **RAG Assistant Queries:** Asking natural language questions like *"Who hasn't logged in this week?"* to get instant lists for follow-up.

The Struggling Student (Indirect Beneficiary)

- **Contextual Support:** Receiving a personalized email when the system detects they are "Drifting" or facing "Concept Confusion".
- **Anonymous Friction Reporting:** Having their repeated chatbot questions flagged to the tutor without the fear of direct monitoring.

4. Strategic Guardrails (Resolving Vulnerabilities)

To address the critical flaws regarding surveillance and AI reliability, TRI NETRA will implement:

- **Privacy-First Telemetry:** Moving away from invasive tab-tracking toward a "weighted engagement index" that rewards active participation over mere "visible" interactions.
- **Human-in-the-Loop AI:** The AI Tutor Assistant will provide "Evidence-Based Insights," linking every suggestion to specific student data points to prevent RAG hallucinations.
- **Authentic Communication:** The AI Email Improver acts as a drafting assistant, ensuring the tutor's unique voice is preserved to maintain the student-teacher bond.

5. Key Success Metrics

- **Time to Intervention:** Reducing the gap between a student getting stuck and receiving tutor support.
- **Course Completion Rate:** Lowering dropout rates by identifying "Drifting" students early.
- **Tutor Efficiency:** Measuring the reduction in manual data-digging through the use of the AI Copilot.

1. Enhanced Intelligence Layer (The "Awareness" Engine)

Your project uses vector embeddings in PostgreSQL to power the AI Copilot. This enables "Evidence-Based Insights" by linking every AI suggestion to specific telemetry data points.

- **RAG Architecture:** Course material and student activity summaries are converted into vector embeddings and stored in PostgreSQL.
- **Learner Diagnostics Logic:**
 - **Attention Drift (Drifting):** Triggered if a student switches tabs more than 5 times or stays idle for 300+ seconds within a 10-minute window.

- **Content Friction (Friction):** Triggered after 3 consecutive quiz failures or 5 repeated views of the same video lesson.
- **OpenAI Integration:** Leverages LLMs to analyze these signals and provide tutors with natural language summaries through the AI Copilot.

2. Frontend & User Experience (The "Visibility" Interface)

The frontend is built for an "Apple-level" experience using **React 18** and **Vite**. It turns raw telemetry into a real-time "Mission Control Center".

- **Fluid Interactions: Framer Motion** and **Lenis** create high-end animations and smooth "scrollytelling" for complex data visualizations.
- **Data Management: TanStack Query** handles real-time background synchronization between the tutor dashboard and the telemetry store.
- **Responsive Layouts:** Uses **Tailwind CSS** and **Radix UI** for the centralized progress table and collapsible AI sidebar.

3. Backend & Data Design (The "Foresight" Infrastructure)

A high-performance server architecture focused on security and data integrity.

- **Data Integrity: Prisma ORM** manages the **PostgreSQL** database, ensuring type-safe storage for student profiles, course metadata, and event logs.
- **Validation: Zod** provides strict schema validation for all incoming student telemetry packets (e.g., video play/pause) to prevent database corruption.
- **Security: JWT and Google Auth** protect sensitive student data from unauthorized access.
- **Automated Action: Nodemailer** sends personalized intervention emails when the system detects a student is "stuck" or "drifting".

4. Strategic Guardrails & Governance

To resolve vulnerabilities in AI reliability and student privacy, TRI NETRA enforces technical limits.

- **Privacy-First Telemetry:** Uses a weighted engagement index instead of invasive constant surveillance.

- **Deep Focus Toggle:** Students can pause non-essential tracking during external research tasks to prevent false "Drifting" flags.
- **Human-in-the-Loop:** The **AI Email Improver** acts only as a drafting assistant; the tutor must manually approve and send all messages to maintain an authentic bond.
- **Data Minimization:** Raw clickstream and navigation logs are purged every 30 days, retaining only high-level progress summaries to protect student privacy.

5. Key API Interface Overview

Endpoint	Method	Purpose
/api/telemetry	POST	Ingests real-time video play/pause and quiz failure events.
/api/diagnostics	GET	Retrieves the list of students flagged as FRICTION or DRIFTING .
/api/ai/query	POST	Powers RAG-based natural language questions for the tutor.
/api/email/improve	POST	Refines tutor message drafts for better tone and clarity.

Product Requirement Document (PRD)

Objective: To provide tutors with "Three Eyes" (Visibility, Awareness, Foresight) to monitor real-time learner progress and intervene proactively.

Target Audience: Online tutors, educational administrators, and data-driven instructors.

Functional Requirements

Advanced Learner Tracking

Centralized Progress Table: Must display student completion percentage, enrollment date, and module-level progress.

Custom Filtering: Tutors must be able to filter by status (Needs Support, On Track, Completed) or custom progress ranges.

Engagement State Categorization: **ENGAGED:** Active interaction with lessons.

DRIFTING: Detected via idle time or tab switching.

FRICITION: Triggered by multiple quiz failures or excessive AI queries.

AI Tutor Assistant (Copilot)

- **RAG-Based Interaction:** Use Retrieval-Augmented Generation to answer natural language questions about course metadata and student activity.
- **Contextual Data Access:** The assistant must index course content, student behavior history, and progress metrics to provide specific insights.
- **Hallucination Check:** Every response must cite specific behavioral data points (e.g., "Student X failed Quiz 2 three times") to ensure factual accuracy.

Communication & Intervention

- **AI Email Improver:** An integrated tool to refine the tone of messages for empathy and clarity before they are sent.
- **Bulk Messaging:** Ability to send direct messages to filtered groups based on progress or engagement status.

- **Critical Alerts:** KPI cards must display high-priority students requiring immediate instructor attention.

1. Advanced User Roles & Permissions

Expanding beyond the Tutor and Student roles, the system requires administrative oversight for organizational management.

- **System Administrator (Super-User):** Must have the authority to manage tutor licenses, configure global course settings, and monitor system-wide health metrics.
- **Institutional Manager:** A role for educational leaders to view aggregated performance data across multiple tutors and cohorts without accessing individual private student-tutor communications.
- **Role-Based Access Control (RBAC):** Every API request must validate the user role via JWT to ensure tutors only access their assigned cohorts.

2. Monetization & Subscription Logic

This section defines the financial flow for the platform, supporting both tutor subscriptions and student course purchases.

- **Tutor Subscription Tiers:** Tutors choose from "Starter," "Professional," or "Enterprise" plans based on the number of active students and AI query limits.
- **Student Payment Gateway:** Secure checkout for students to purchase course access using Stripe or Razorpay.
- **Enrollment Trigger:** The system must automatically update the student's status to "Active" and grant access to module metadata immediately after payment verification.

3. Third-Party Ecosystem Integration

To resolve the "Information Silos" problem, the platform must connect with existing educational tools.

- **LMS Synchronization:** Provide API hooks to import student rosters and curriculum structures from Learning Management Systems like Moodle or Canvas.
- **External Content Indexing:** The RAG assistant must be able to index external research links shared by tutors to provide broader contextual insights.

4. Detailed Notification & Preference Logic

This adds granularity to the "Communication & Intervention" requirements.

- **Trigger Management:** Tutors must be able to toggle which events generate an automated email versus a dashboard "Critical Alert".
- **Threshold Customization:** Tutors can define the specific number of quiz failures or idle minutes that trigger a **FRICTION** or **DRIFTING** state.
- **Notification Frequency:** Provide "Instant," "Daily Digest," or "Weekly Summary" options for tutor alerts to prevent dashboard fatigue.

5. Reporting & Data Portability

Tutors require offline access to the "Visibility" data for institutional compliance.

- **CSV/PDF Export:** Enable tutors to download individual student performance timelines and cohort-wide progress summaries.
- **Intervention Logs:** A dedicated report detailing all sent emails and AI-improved messages to track tutor efficiency.

6. Onboarding & Lifecycle Management

This defines the start and end states for users on the platform.

- **Course Creator Wizard:** A step-by-step interface for tutors to upload module metadata and configure telemetry tracking.
- **Student Invitation System:** Bulk invite students via email with unique registration links that automatically map them to specific cohorts.
- **Course Completion Flow:** Automated process to mark a student as "Completed" once module-level progress reaches 100%, triggering a completion certificate.

Non-Functional Requirements

Security & Privacy

- **Data Minimization:** Anonymize all behavioral telemetry at rest; purge raw clickstream data after 30 days.
- **IAM Integration:** Use Identity and Access Management (IAM) for all cloud resource access; strictly forbid the use of local .env files for secrets in production.

Privacy Toggle: Provide a "Deep Focus" mode for students to pause tab-tracking during external research.

Performance & Reliability

- **Latency:** Live activity feeds must have a delay of less than 30 seconds to be considered "real-time".
- **Scalability:** The system must support simultaneous monitoring of up to 5,000 active students per course.
- **AI Uptime:** The RAG-based assistant must maintain 99.5% availability during core tutoring hours.

4. User Stories	User	Role	Requirement	Goal
Story ID				
US-01		Tutor	"Show me all students who failed Module 2."	To identify a cohort for a live review session.
US-02		Tutor	"Improve the tone of this warning email."	To sound supportive rather than punitive.
US-03		Student	"Enable Deep Focus mode."	To conduct external research without lowering engagement scores.

Accessibility Standards (Non-Functional)

To support diverse learners, the platform must adhere to global accessibility guidelines.

- **WCAG 2.1 Compliance:** The React frontend must meet Level AA standards, including keyboard navigation support and high-contrast UI modes.
- **Screen Reader Compatibility:** Ensure all data tables and KPI cards utilize appropriate ARIA labels for assistive technology users.

Use Case

1. Platform Actors

- **Primary Actor:** Tutor (Instructor/Admin).
- **Secondary Actor:** Student (Learner).
- **Administrative Actor:** System Administrator (Platform Owner).
- **System Actor:** AI Copilot (RAG-based Assistant).

2. Instructional & Monitoring Use Cases

UC-01: Early Detection of Struggling Students

- **Goal:** Identify students likely to fail before it happens.
- **Basic Flow:**
 1. System analyzes real-time activity data.
 2. Algorithm flags student as **FRICTION** after three consecutive quiz failures.
 3. Tutor Dashboard updates the Critical Alerts KPI card.
 4. Tutor clicks alert to view the Struggle Diagnosis.

UC-02: Natural Language Insight Retrieval (AI Copilot)

- **Goal:** Allow tutors to query complex student data using natural language.
- **Basic Flow:**
 1. Tutor opens the AI Tutor Assistant chat sidebar.
 2. Tutor queries: "Which students in Module 3 are drifting?".
 3. AI Copilot retrieves metadata from the database.
 4. System provides summarized list with deep-dive links.

UC-03: Concept Confusion Analysis

- **Goal:** Identify course topics causing friction across the class.

- **Basic Flow:**

1. System aggregates chatbot conversation logs.
2. Chatbot Stats card highlights a spike in specific module queries.
3. Tutor views "Confusion Patterns" to see common questions.
4. Tutor schedules a live review session for that topic.

3. Lifecycle & Financial Use Cases

UC-04: Course Enrollment & Payment

- **Goal:** Enable students to purchase access and trigger automated enrollment.

- **Basic Flow:**

1. Student selects a course on the platform.
2. Student completes payment via the integrated gateway.
3. System verifies transaction and updates the enrollment database.
4. Student receives immediate access to module metadata and lessons.

UC-05: Course Completion & Reporting

- **Goal:** Mark students as completed and generate performance summaries.

- **Basic Flow:**

1. Student reaches 100% progress in all modules.
2. System updates student status to **COMPLETED**.
3. Tutor views the final performance timeline in the deep-dive view.
4. System generates a completion report for institutional records.

4. Administrative & Operational Use Cases

UC-06: Institutional Setup & License Management

- **Goal:** Enable platform owners to manage tutor accounts and system health.

- **Basic Flow:**

1. Administrator logs into the Super-User Dashboard.

2. Administrator creates a new tutor account and assigns a student volume license.
3. Administrator monitors global system latency to ensure real-time feed performance.

UC-07: LMS Data Synchronization

- **Goal:** Automate the import of rosters and course data from external tools.
- **Basic Flow:**
 1. Tutor selects "Import from LMS" in the course creator wizard.
 2. System connects to the external LMS API.
 3. System synchronizes student names, IDs, and existing progress metrics.
 4. TRI NETRA begins tracking new telemetry on the imported cohort.

5. Privacy & Configuration Use Cases

UC-08: Student Privacy (Deep Focus Mode)

- **Goal:** Allow external research without lowering engagement scores.
- **Basic Flow:**
 1. Student toggles Deep Focus Mode on the interface.
 2. System pauses "Attention Drift" telemetry tracking.
 3. Student conducts research and then toggles the mode off.

UC-09: Threshold & Preference Customization

- **Goal:** Allow tutors to define their own alert parameters.
- **Basic Flow:**
 1. Tutor opens the "Monitoring Preferences" menu.
 2. Tutor adjusts the **FRICTION** trigger from three quiz failures to five.
 3. Tutor configures the system to send instant emails for **FRICTION** but only dashboard alerts for **DRIFTING**.

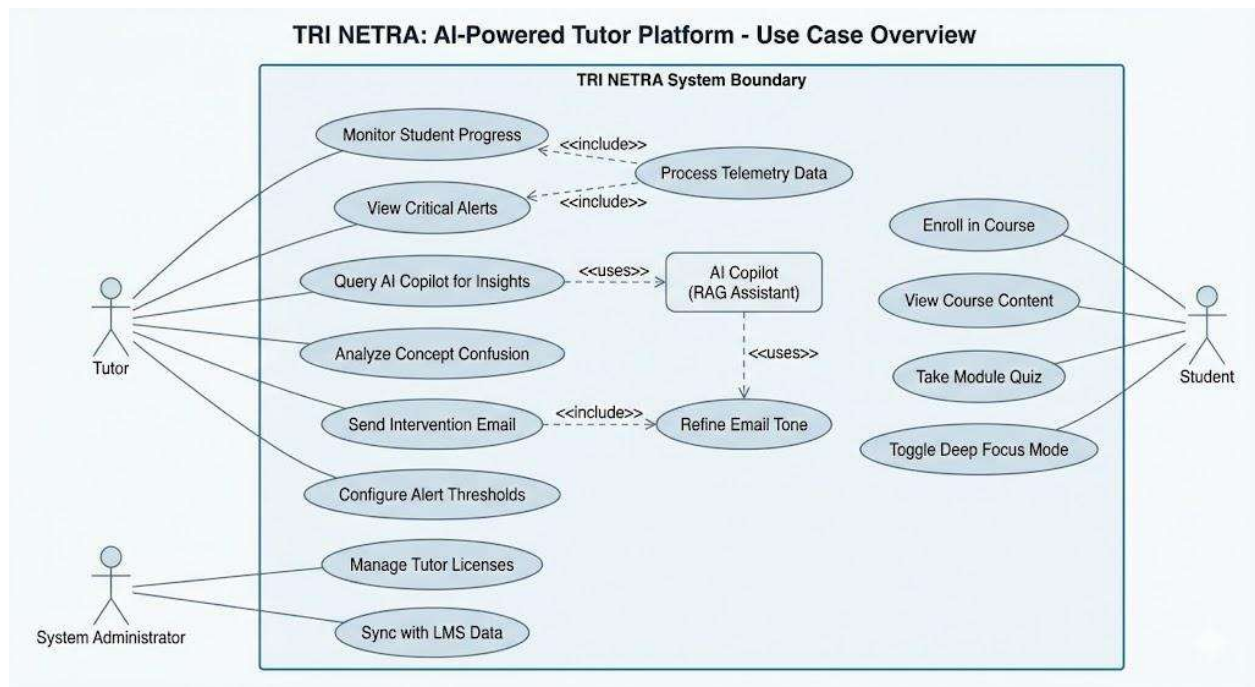
6. Communication Use Cases

UC-10: Proactive Intervention via AI Email

- **Goal:** Send personalized messages to at-risk students.

- **Basic Flow:**

1. Tutor selects a filtered group from the progress table.
2. Tutor writes a draft message.
3. Tutor uses AI Email Improver to refine tone.
4. Tutor reviews and sends the intervention.



System Design

Project Vision: An AI-powered "Mission Control Center" designed to provide tutors with deep visibility, early awareness, and proactive foresight into student learning behaviors.

1. System Architecture Overview

TRI NETRA utilizes a modular, cloud-native architecture designed to process high-volume student telemetry and provide real-time AI-driven insights. The system is divided into three primary layers: the Presentation Layer, the Intelligence & Analytics Layer, and the Data Persistence Layer.

Presentation Layer (Tutor Dashboard)

- **Centralized Progress Table:** A core UI component displaying student names, module completion percentages, and real-time learning status.
- **AI Copilot Sidebar:** A conversational interface for tutors to query student data using natural language.
- **Live Activity Feed:** A real-time timeline of events, including lesson views, video interactions (play/pause), and quiz attempts.
- **KPI Summary Cards:** High-level visual metrics for active learners, average cohort progress, and critical alerts.

Intelligence & Analytics Layer

- **Struggle Diagnosis Engine:** A rule-based and algorithmic engine that analyzes telemetry to categorize students into engagement states: **ENGAGED**, **DRIFTING**, or **FRICTION**.
- **RAG AI Assistant:** A Retrieval-Augmented Generation pipeline that indexes course content and student metadata to answer complex pedagogical queries.
- **Chatbot Conversation Analysis:** A specialized module that "listens" to student-chatbot interactions to detect "Concept Confusion" or "Blocked Learning" across a cohort.

2. Functional Module Design

Progress Tracking & Monitoring

- **Filtering & Sorting:** Tutors can filter learners by status (e.g., Needs Support, On Track) or sort them by completion metrics.

- **Cohort Isolation:** Enables tutors to switch between different student batches to compare performance across groups.

Real-Time Behavioral Telemetry

Engagement Indicators: **Attention Drift:** Triggered by signals like switching tabs or repeated idle time.

Content Friction: Triggered by multiple quiz failures or repeated viewing of the same content.

Student Deep Dive: Provides a granular view of an individual student's activity history and interaction patterns.

Communication & Intervention

- **Direct Messaging:** Support for sending individual or bulk emails directly from the dashboard.
- **AI Email Improver:** Uses natural language processing to refine the tone and clarity of tutor messages before transmission.

3. Data Design & Storage

Relational Data (Structured)

- **Student Profiles:** Stores core identity information and enrollment status.
- **Course Metadata:** Includes module structures, quiz banks, and lesson hierarchies.

Telemetry Store (Unstructured)

- **Event Logs:** Captures high-frequency data such as clickstream events, video timestamps, and navigation patterns.

Vector Knowledge Base (AI Context)

- **Contextual Embeddings:** Stores vectorized representations of course materials and historical student behavior logs to support the RAG-based AI Assistant.

4. Security, Privacy, and Reliability Safeguards

To address potential vulnerabilities, the system implements several technical guardrails:

- **Evidence-Based RAG:** The AI Copilot is configured to cite specific behavioral data points for every insight provided, minimizing the risk of hallucinations.

- **Data Minimization:** Behavioral telemetry is categorized by severity; raw logs are purged according to a defined retention policy to reduce surveillance risks.
- **Deep Focus Mode:** A student-facing privacy toggle that pauses non-essential behavioral tracking (like tab-switching) during external research tasks.
- **IAM-First Access:** Secure integration using Identity and Access Management (IAM) to manage system permissions and prevent unauthorized data access.

5. Deployment and Infrastructure

- **Real-Time Processing:** Utilizes event-driven triggers to update "Critical Alerts" on the dashboard in under 30 seconds.
- **Scalability:** Designed to handle multi-course support, allowing tutors to manage large student populations across different subjects.

1. Missing Technology Specifications

The document describes the architecture in broad terms but lacks the specific toolset identified in the project's technical stack:

- **Missing Toolset:** There is no mention of **Node.js/Express** for the server, **React 18** for the frontend, or **Prisma ORM** for database management.
- **Interface Libraries:** Specific libraries for high-end UX, such as **Framer Motion**, **Lenis**, and **Radix UI**, are omitted from the design overview.

2. Database Schema and Entity Relationships

While the document defines data storage categories (Relational, Telemetry, Vector), it lacks structural details:

- **Schema Details:** There is no Entity Relationship Diagram (ERD) or table definition for core entities like Student, Cohort, CourseMetadata, or TelemetryEvent .
- **Vector Implementation:** The document mentions a vector knowledge base but doesn't specify using **PostgreSQL** with vector embeddings for RAG.

3. API and Communication Layer

The design identifies the layers of the system but omits the "glue" between them:

- **API Specification:** There is no definition of REST or GraphQL endpoints for actions like retrieving learner diagnostics, ingesting telemetry, or triggering AI assistant queries.

- **Validation Logic:** The use of **Zod** for strict schema validation of API inputs and outputs is missing from the security and reliability section.

4. Diagnostic Algorithm Thresholds

The document mentions a "rule-based" Struggle Diagnosis Engine but fails to define the actual rules:

- **Missing Triggers:** It identifies "Attention Drift" and "Content Friction" as indicators but does not specify the exact thresholds (e.g., number of tab switches or quiz failures) required to trigger these alerts .

5. Operational and Deployment Lifecycle

The section on deployment is high-level and lacks actionable procedures:

- **CI/CD Pipeline:** There is no detail on the continuous integration/deployment flow, environment separation (Staging vs. Production), or automated testing .
- **Secret Management:** While mentioning IAM, it does not explicitly forbid local .env files in production or define a strategy for managing API keys for OpenAI integration.

6. Authentication and Identity Flow

- **Security Gaps:** The document mentions IAM for cloud access but omits the user-facing security strategy, such as **JWT and Google Auth** for secure tutor sessions.

High-Level Architecture (HLA)

1. Presentation Layer (Fluid Visibility)

The interface is designed for high-end "Apple-level" performance with real-time synchronization.

- **Tutor Dashboard (React 18 + Vite):** A single-page application using **TanStack Query** to sync the Centralized Progress Table and KPI cards with the backend in under 30 seconds.
- **AI Copilot Sidebar:** A RAG-powered chat interface that allows tutors to ask natural language questions about student metadata.
- **Animations & Motion:** Utilizes **Framer Motion** and **Lenis** for smooth interactions and scrollytelling data views.
- **Component Foundation:** Built using **Radix UI** and **Tailwind CSS** for accessibility and modern styling.

2. Intelligence & Analytics Layer (Awareness & Logic)

This layer processes telemetry and powers the "Three Eyes" of awareness and foresight.

- **Struggle Diagnosis Engine:** A logic service within the Node.js backend that analyzes incoming events to assign labels: **ENGAGED**, **DRIFTING**, or **FRICTION**.
- **RAG AI Assistant (Node.js + OpenAI):** An orchestrator that retrieves relevant context from the PostgreSQL vector store to ground AI responses in factual behavioral data.
- **Concept Confusion Module:** Aggregates chatbot logs to detect cohort-level learning gaps and "Blocked Learning" signals.

3. Unified Data Layer (Foresight & Persistence)

The platform uses **PostgreSQL** as a multi-model store for all structured, telemetry, and vector data.

- **Relational Database (PostgreSQL):** Stores student profiles, course metadata, and cohort enrollment history.

- **Telemetry Event Store:** Captures high-frequency clickstream data (video play/pause, tab switches).
- **Vector Store (pgvector):** Stores embeddings of course content and student history to support semantic search in the RAG assistant.
- **Prisma ORM:** Provides type-safe access to all data models, ensuring integrity and automated migrations.

4. Security, Integration, & Deployment

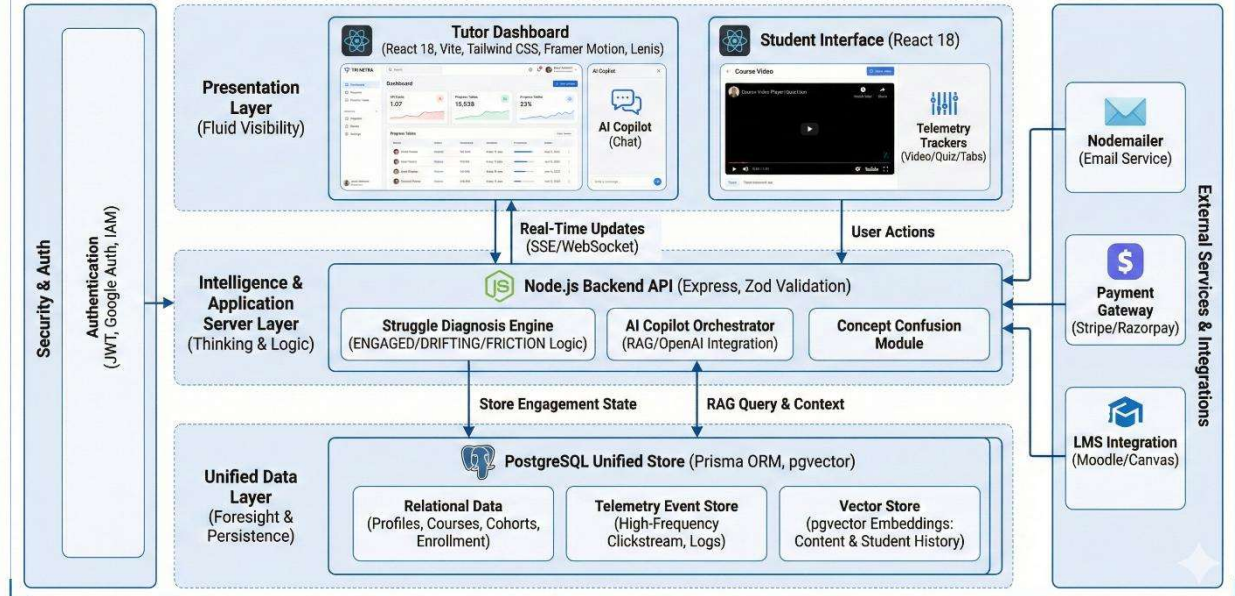
Technical guardrails ensure reliability and prevent surveillance vulnerabilities.

- **Security & Auth:** Employs **JWT** and **Google Auth** for secure tutor sessions and social login.
- **Validation:** Uses **Zod** for strict schema validation of all API inputs and telemetry packets.
- **Communications:** Integrates **Nodemailer** to handle automated critical alerts and AI-improved email drafts.
- **Model Context Protocol (MCP):** Connects AI agents to private data sources without exposing raw database credentials.

Updated System Data Flow

Step	Component	Action
1. Sensing	Student UI	Captures clickstream, video play/pause, and quiz failures.
2. Thinking	Node.js Backend	Analyzes patterns via Zod and assigns engagement labels.
3. Storing	PostgreSQL	Persists both structured telemetry and vector embeddings.
4. Inquiring	AI Copilot	Retrieves context from pgvector to answer tutor questions.
5. Resolving	Email Service	Sends personalized support drafts via Nodemailer for tutor approval.

TRI NETRA: High-Level Architecture (Sensing-Thinking-Acting Cycle)



Data Design Document

1. Unified PostgreSQL Strategy

PostgreSQL serves as the single source of truth for the platform. Using a single database reduces infrastructure complexity while supporting high-concurrency telemetry and AI-driven insights.

- **Relational Storage:** Manages structured entities like users, courses, and cohorts.
- **Telemetry Event Store:** Stores high-frequency clickstream data such as video play and quiz attempts.
- **Vector Store (pgvector):** Stores high-dimensional embeddings for the RAG-based AI Copilot to answer questions about course material.

2. Core Relational Entities

Prisma ORM manages these models to ensure type safety and automated migrations.

- **User:** Profiles for Tutors and Students with unique identifiers and enrollment dates.
- **Course & Module:** Curriculum hierarchy tracking titles and lesson counts.
- **Cohort:** Batch groupings to compare performance across different student groups.
- **Enrollment:** Links students to courses and tracks completion percentages.

3. Telemetry & Engagement Logic

The system captures granular actions to feed the "Mission Control" live feed.

- **Activity Logs:** Tracks video play/pause, timestamps, and navigation patterns.
- **Engagement State:** Records calculated labels including **ENGAGED**, **DRIFTING**, and **FRICTION**.
- **Assessment Logs:** Stores quiz results and specific questions causing friction.

4. Data Flow & Zod Validation

The system uses **Zod** for strict schema validation of all incoming telemetry to prevent database corruption.

1. **Ingestion:** Student actions generate event packets on the interface.
2. **Validation:** Zod verifies the packet structure before the backend processes it.
3. **Processing:** The diagnosis engine checks for friction thresholds, such as three consecutive quiz failures.
4. **Persistence:** Validated events are stored in PostgreSQL for real-time dashboard updates.

5. Security & Privacy Design

Technical guardrails resolve surveillance and data exfiltration risks.

- **Deep Focus Mode:** When active, the system halts tab-switching and idle-time tracking in the event store.
- **Anonymization:** Student IDs are masked during cohort-level confusion checks.
- **Retention Policy:** Raw clickstream data is purged after 30 days to protect student privacy.
- **IAM-First Access:** Database connection strings and AI API keys are managed through secure identity-based access.

API Design Document

1. Base Configuration

- **Base URL:** https://api.trinetra.io/v1.
- **Authentication:** Bearer Token (JWT) via IAM.
- **Content Type:** application/json.
- **Validation:** All inputs are strictly validated via **Zod** schemas before processing.

2. Authentication & Identity Endpoints

Endpoint	Method	Purpose
/auth/login	POST	Exchanges credentials for a JWT.
/auth/google	POST	Handles social login and account creation.
/students/me/privacy	PATCH	Toggles Deep Focus Mode for student privacy.

3. Administrative & Onboarding Endpoints

Endpoint	Method	Purpose
/courses	POST	Creates new course metadata and module structures.
/cohorts	POST	Groups students into new batch units.
/invitations	POST	Executes bulk student invites and registration links.
/settings/thresholds	PATCH	Defines custom triggers for FRICTION and DRIFTING states.

4. Learner & Progress Endpoints

Endpoint	Method	Purpose
/learners	GET	Lists students with filters for status and progress.
/learners/{id}/drill-down	GET	Fetches a granular activity timeline for deep dives.

Endpoint	Method	Purpose
/analytics/export	GET	Generates a CSV/PDF report of cohort analytics.

5. Telemetry & Activity Endpoints

Endpoint	Method	Purpose
/telemetry/events	POST	Ingests real-time actions (video pause, quiz fail).
/activity/live-feed	GET	Streams the real-time event timeline via SSE .

6. AI Assistant & Communication Endpoints

Endpoint	Method	Purpose
/ai/assistant/query	POST	Entry point for the RAG-based natural language chat.
/ai/email/improve	POST	Refines tutor message tone for empathy and clarity.
/communications/bulk	POST	Executes mass messaging to a filtered student group.

7. Financial & Integration Endpoints

Endpoint	Method	Purpose
/payments/checkout	POST	Initiates a Stripe/Razorpay session for course purchase.
/integrations/lms/sync	POST	Pulls rosters and module data from Moodle or Canvas.

8. Analytics & KPI Endpoints

Endpoint	Method	Purpose
/analytics/kpis	GET	Data for Active Learners and Critical Alert cards.
/analytics/chatbot/confusion	GET	Aggregates logs to detect cohort-level learning gaps.

9. Error Handling

- **400 Bad Request:** Invalid Zod validation or query parameters.
- **401 Unauthorized:** Missing or expired JWT.
- **429 Too Many Requests:** AI query rate limit exceeded.